

1–[55%] Construa uma classe *Sequências* que permite armazenar sequências de valores genéricos comparáveis entre si. A classe deve ter as seguintes características e métodos, respeitando a complexidade indicada:

- *criaSequência (nome)* : Cria uma nova sequência com o nome indicado. Complexidade $O(1)$.
- *acrescenta (nome, valor)* : acrescenta o valor à sequência com o nome indicado. Complexidade $O(1)$.
- *getSequencia (nome)* : devolve uma List com a sequência com o nome indicado. Complexidade $O(1)$.
- *ordenada (nome)* : verifica se a sequência com o nome indicado está ou não ordenada. Complexidade $O(N)$.
- A classe *Sequências* deve ser iterável. O iterador deve percorrer todos os valores de todas as sequências armazenadas. A remoção de valores através do iterador não é suportada. Deve gerar a excepção *ConcurrentModificationException* de forma adequada sempre que necessário.

2- [10%] Assuma que dispõe de um método *int pesquisa (int m[], int valor)* que efectua uma pesquisa binária. Este método devolve a posição em que o valor procurado se encontra, ou então um valor negativo (-X) caso este não esteja no array indicado. O valor de $\text{abs}(X+1)$ indica uma posição em que o valor procurado poderia ser inserido para preservar a ordem. **Este método já existe e não precisa de o fazer.** Construa um método que recebe um array de inteiros ordenado sem valores duplicados e um inteiro *N*. Este método deve devolver o valor do array com valor mais próximo de *N*. Em caso existam várias opções, qualquer um dos valores possíveis pode ser devolvido. **O método deve ter complexidade logarítmica.**

Por exemplo, se o array for $A=\{1,3,10,15\}$, então *maisProximo(A,4)* deve devolver 3, *maisProximo(A,10)* deve devolver 10, *maisProximo(A,100)* deve devolver 15 e *maisProximo(A,2)* pode devolver 1 ou 3.

3–[35%] Considere uma classe *Arvore* que implementa uma árvore binária.

a) Crie um método recursivo *mostraIntervalo* que recebe dois valores (que definem os limites de um intervalo) e imprime todos os valores da árvore dentro do intervalo especificado. O método não deve realizar travessias da árvore desnecessárias. Construa também o respetivo método público não-recursivo.

b) Crie um método recursivo *maiores* que recebe um valor e que devolve uma lista com os valores da árvore maiores do que esse valor. Crie também o respetivo método público. ***Não deve acrescentar outros parâmetros ao método, para além de um nodo da árvore e do valor em questão, nem usar valores estáticos.***

c) Crie um método recursivo *removeSubÁrvore* que recebe um valor e remove toda a sub-árvore com raiz nesse valor. Crie também o respetivo método público.

